

Project Report Format

Contents

- 1. INTRODUCTION**
 - 1.1 Project Overview
 - 1.2 Purpose
- 2. IDEATION PHASE**
 - 2.1 Problem Statement
 - 2.2 Empathy Map Canvas
 - 2.3 Brainstorming
- 3. REQUIREMENT ANALYSIS**
 - 3.1 Customer Journey map
 - 3.2 Solution Requirement
 - 3.3 Data Flow Diagram
 - 3.4 Technology Stack
- 4. PROJECT DESIGN**
 - 4.1 Problem Solution Fit
 - 4.2 Proposed Solution
 - 4.3 Solution Architecture
- 5. PROJECT PLANNING & SCHEDULING**
 - 5.1 Project Planning
- 6. FUNCTIONAL AND PERFORMANCE TESTING**
 - 6.1 Performance Testing
- 7. RESULTS**
 - 7.1 Output Screenshots
- 8. ADVANTAGES & DISADVANTAGES**
- 9. CONCLUSION**
- 10. FUTURE SCOPE**
- 11. APPENDIX**
 - Source Code(if any)
 - Dataset Link
 - GitHub & Project Demo Link

1. Introduction

1.1 Project Overview

The A Comprehensive Measure of Well-Being – Human Development Index (HDI) Prediction System is an Artificial Intelligence and Machine Learning-based web application developed to predict the Human Development Index (HDI) of a country using important socio-economic indicators. The Human Development Index is one of the most widely used measures introduced by the United Nations Development Programme (UNDP) to evaluate the overall development and quality of life of a country's population.

Technologies Used

Layer	Technology
Programming Language	Python 3.x
Backend Framework	Flask
Frontend	HTML5, CSS3, JavaScript, Bootstrap
Machine Learning	Scikit-learn
Algorithm	Linear Regression
Data Processing	Pandas, NumPy
Visualization	Matplotlib
Database	SQLite
Model Storage	Pickle (.pkl)
Development Environment	Visual Studio Code
Version Control	Git & GitHub

1.2 Purpose

The primary purpose of the A Comprehensive Measure of Well-Being – Human Development Index (HDI) Prediction System is to automate the prediction of Human Development Index values using Machine Learning techniques. Traditionally, HDI calculations require extensive statistical analysis and manual computation of health, education, and income indices. This process is time-consuming, especially when analyzing multiple countries or future development scenarios.

2. Ideation Phase

2.1 Problem Statement

The Human Development Index (HDI) is one of the most widely accepted indicators for measuring the overall development of a country. It evaluates development based on three fundamental dimensions: health, education, and standard of living. Governments, researchers, and international organizations use HDI to compare the development status of different countries and to formulate effective public policies. Traditionally, HDI is calculated manually using statistical formulas that require collecting large volumes of socio-economic data such as life expectancy, years of schooling, and gross national income per capita. This process is time-consuming and is prone to human error when performed manually.

2.2 Empathy Map Canvas

User Persona: Government Policy Analyst / Researcher

Think & Feel

- Need accurate HDI predictions for development planning.
- Want to analyze how education, health, and income affect human development.
- Need reliable reports for policy formulation.
- Prefer a simple system that reduces manual calculations.
- Want quick predictions without statistical complexity.

Hear

- Government initiatives for improving quality of life.
- Recommendations from economists and development experts.
- UNDP reports on sustainable development.
- Public demand for better education, healthcare, and income opportunities.

See

- Large datasets containing development indicators.
- Different development levels among countries.
- Time-consuming manual analysis.
- Increasing demand for AI-based policy support systems.

Say & Do

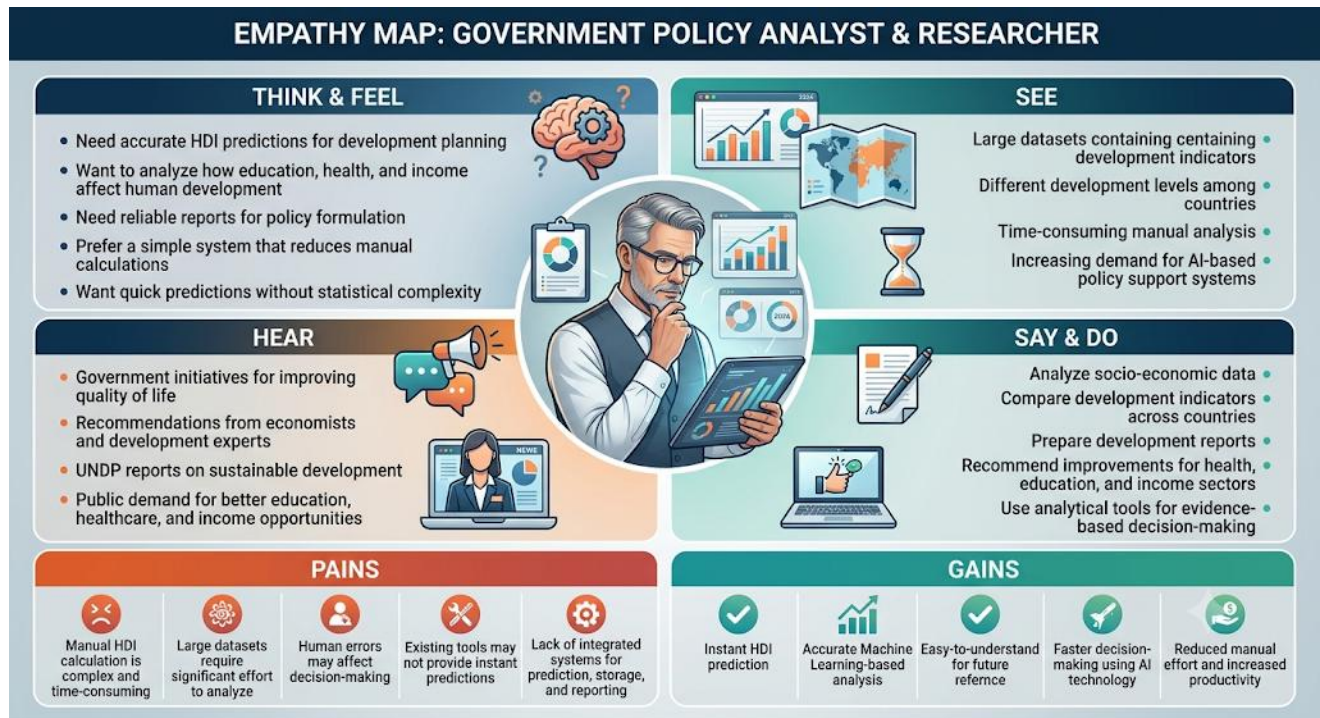
- Analyze socio-economic data.
- Compare development indicators across countries.
- Prepare development reports.
- Recommend improvements for health, education, and income sectors.
- Use analytical tools for evidence-based decision-making.

Pain

- Manual HDI calculation is complex and time-consuming.
- Large datasets require significant effort to analyze.
- Human errors may affect decision-making.
- Existing tools may not provide instant predictions.
- Lack of integrated systems for prediction, storage, and reporting.

Gain

- Instant HDI prediction.
- Accurate Machine Learning-based analysis.
- Easy-to-understand graphical reports.
- Prediction history for future reference.
- Faster decision-making using AI technology.
- Reduced manual effort and increased productivity.

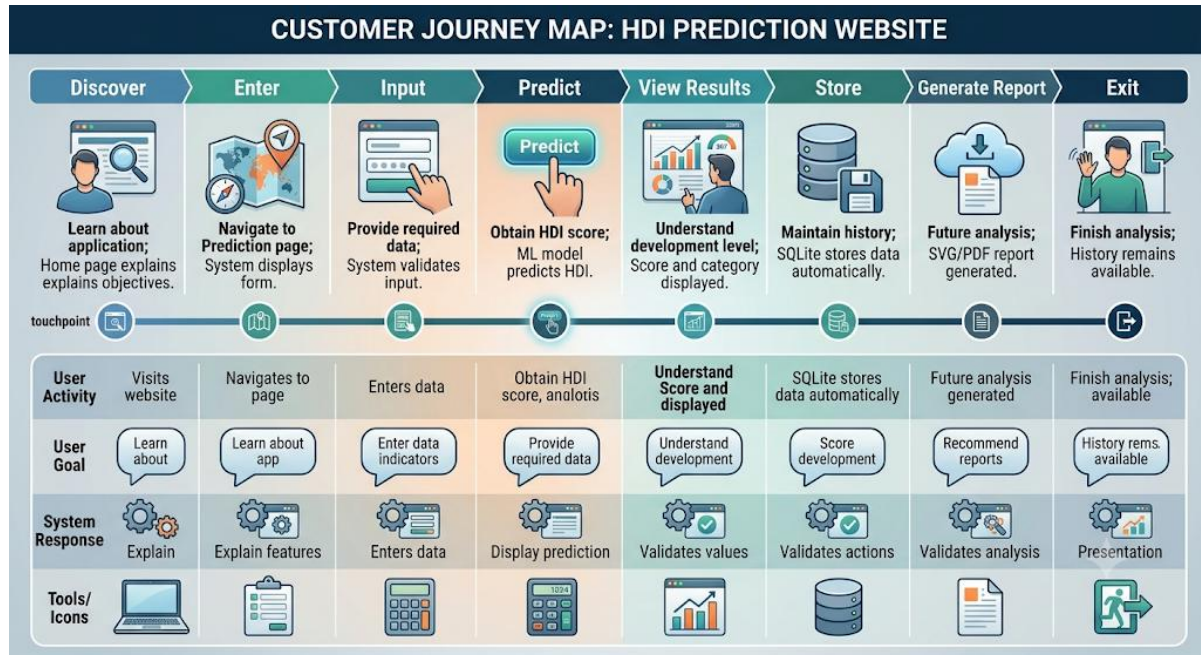


2.3 Brainstorming

- Human Development Index Prediction
- Country Development Classification
- Education Impact Analysis
- Health Index Prediction
- Income-Based Development Analysis
- Sustainable Development Dashboard
- Human Development Trend Analysis
- AI-Based Development Recommendation System
- Interactive Country Comparison Dashboard
- Automated Development Report Generation

3. Requirement Analysis

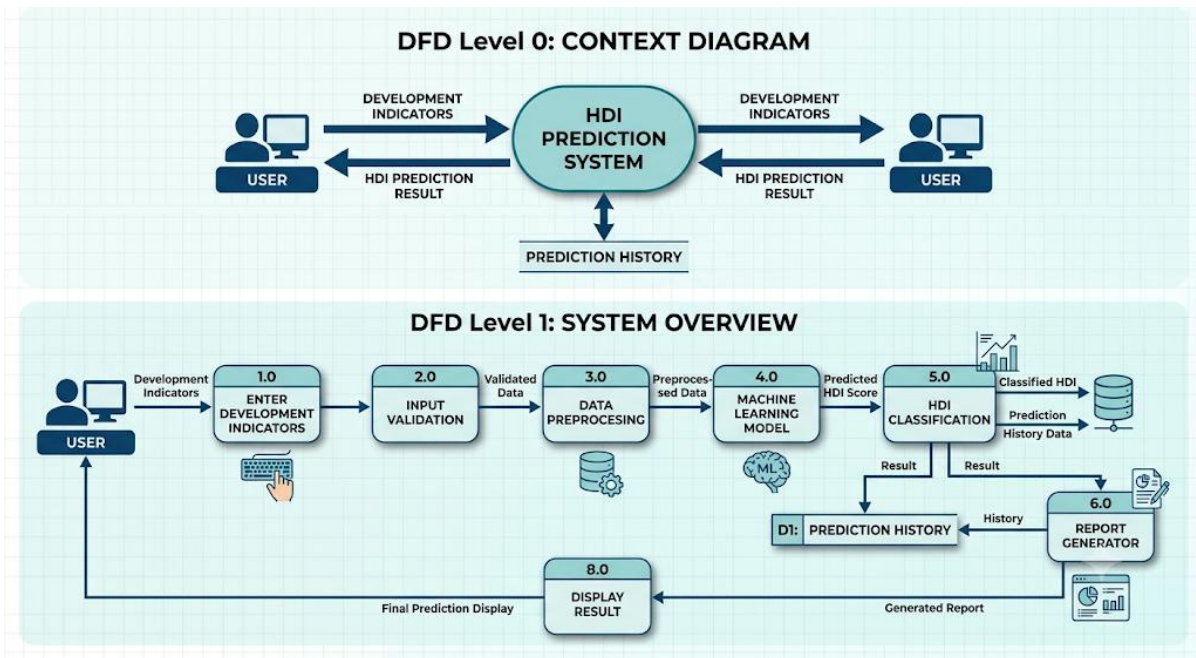
3.1 Customer Journey Map



3.2 Solution Requirements

- **Functional Requirements**
 - User Input Management
 - Input Validation
 - Data Preprocessing
 - Machine Learning Prediction
 - HDI Classification
 - Database Operations
 - Report Generation
 - Prediction History
 - Visualization
- **Non-Functional Requirements**
 - Security
 - Reliability
 - Performance
 - Scalability
 - Availability
 - Maintainability

3.3 Data Flow Diagram



3.4 Technology Stack

Layer	Technology	Purpose
Programming Language	Python 3.x	Backend development
Web Framework	Flask	Develop web application
Frontend	HTML5	User Interface
Styling	CSS3	Application styling
Scripting	JavaScript	Client-side validation
Machine Learning	Scikit-learn	Train prediction model
Algorithm	Linear Regression	Predict HDI
Data Processing	Pandas	Dataset handling
Numerical Computing	NumPy	Mathematical operations
Visualization	Matplotlib	Charts and analysis
Database	SQLite	Store prediction history
Model Storage	Pickle (.pkl)	Save trained model
IDE	Visual Studio Code	Development environment
Version Control	Git & GitHub	Source code management
Operating System	Windows/Linux	Application deployment

4. PROJECT DESIGN

4.1 Problem Solution Fit

Customer Segments

- Government Officers
- Researchers
- Students
- Educational Institutions
- NGOs
- Policy Makers

Problem

Calculating the Human Development Index (HDI) manually is difficult and time-consuming. It requires collecting data, performing calculations, and analyzing multiple development indicators.

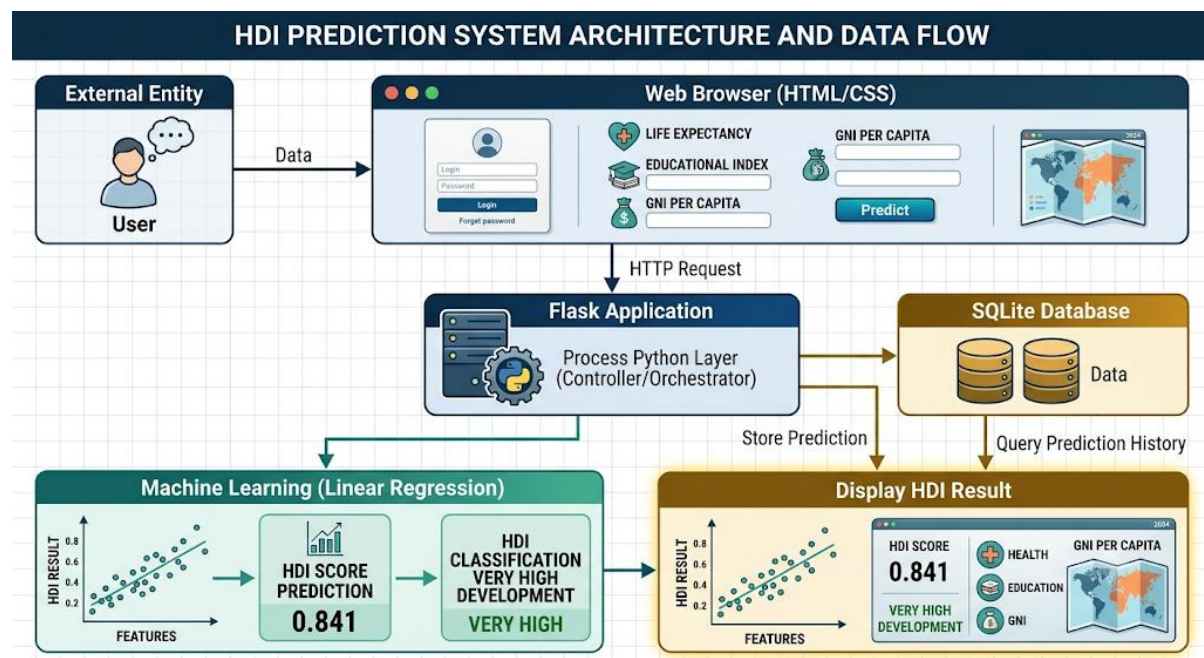
Solution

The proposed system predicts the Human Development Index automatically using Machine Learning.

4.2 Proposed Solution

The proposed solution is a Machine Learning-based HDI Prediction System developed using Python and Flask.

4.3 Solution Architecture



5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

Sprint-1

- Literature Survey
- Dataset Collection
- Exploratory Data Analysis (EDA)
- Data Preprocessing
- Feature Selection

Sprint-2

- Train-Test Data Split
- Linear Regression Model Development
- Model Training
- Model Evaluation
- Save Trained Model (HDI.pkl)

Sprint-3

- Flask Web Application Development
- Home Page Development
- Prediction Form Development
- Result Page Development
- SQLite Database Integration

Sprint-4

- Model Integration with Flask
- Prediction History Module
- Report Generation
- Functional Testing
- Performance Testing
- Deployment

Velocity

- Total Story Points = 44
- Total Duration = 8 Days
- Average Velocity = 5.5 Story Points/Day

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

Functional Tests

- Input Validation
- Data Preprocessing Validation
- HDI Prediction Validation

- HDI Category Classification
- Model Loading
- Database Operations
- Report Generation
- Prediction History

Performance Tests

- Response Time Test
- Model Prediction Speed
- Concurrent Prediction Requests
- Database Performance Test
- Application Load Test
- Report Generation Speed
-

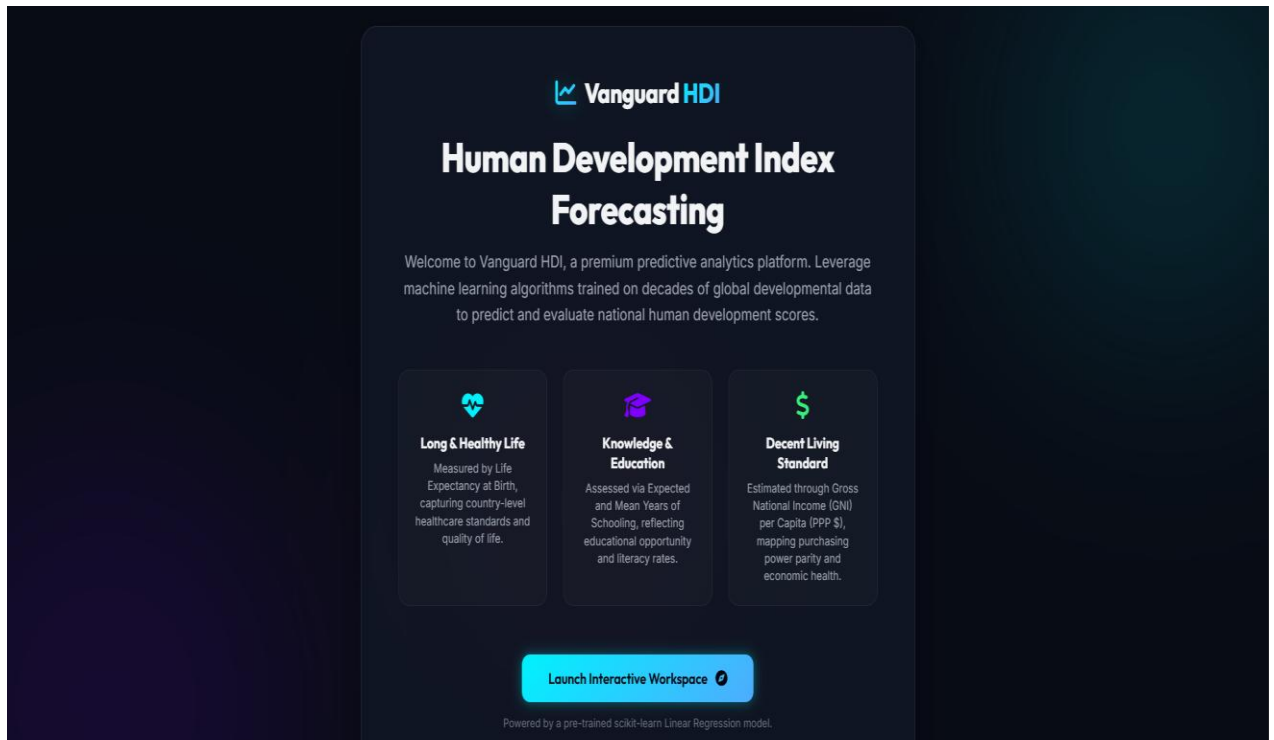
Result

- Passed Functional Tests: 8/8
- Passed Performance Tests: 6/6

7. RESULTS

7.1 Output Screenshots

Home Page



Login Form



Access Predictive Workspace

Enter your details to initiate an analysis session.

FULL NAME

 e.g. Alice Smith

WORK EMAIL

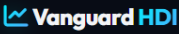
 name@wellbeing.org

PROFESSIONAL ROLE

 Global Policy Analyst 

Initialize Session 

Loan Prediction Form



● Session Active  pallavi Viewer 

Prediction Configurator

Input development metrics to calculate human well-being indices.

SELECT TARGET COUNTRY (REFERENCE TEMPLATE)

 Select country... 

LIFE EXPECTANCY AT BIRTH 70.0 yrs

20.0 yrs

90.0 yrs

MEAN YEARS OF SCHOOLING 7.0 yrs

0.0 yrs

15.0 yrs

EXPECTED YEARS OF SCHOOLING 10.0 yrs

0.0 yrs

18.0 yrs

GNI PER CAPITA (PPP \$)

\$

15000

Range: \$100 to \$100,000. Expressed in Purchasing Power Parity (PPP) USD.

MACHINE LEARNING ENGINE

 RandomForestRegressor (HDI Score Predictor) 

Random Forest Regressor Accuracy Score: 0.985 R² Score: 0.978

Compute HDI & Report 

Visualization Report

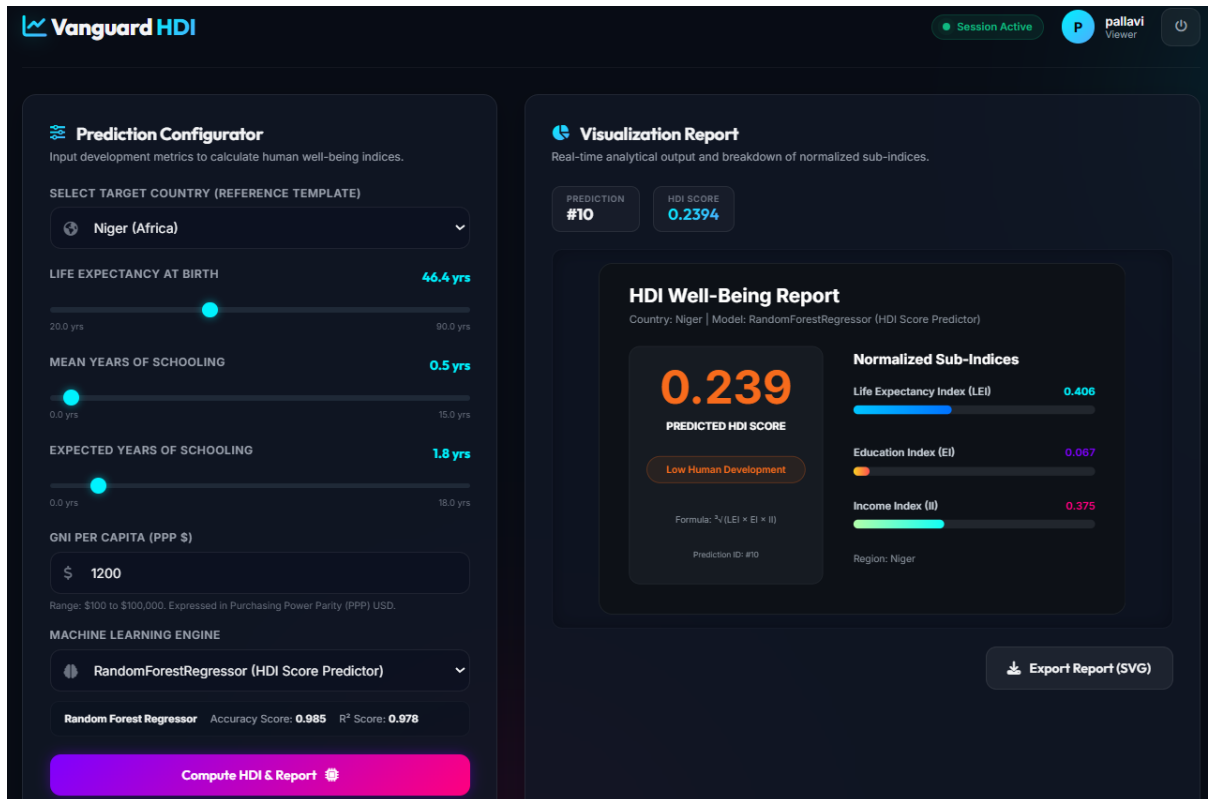
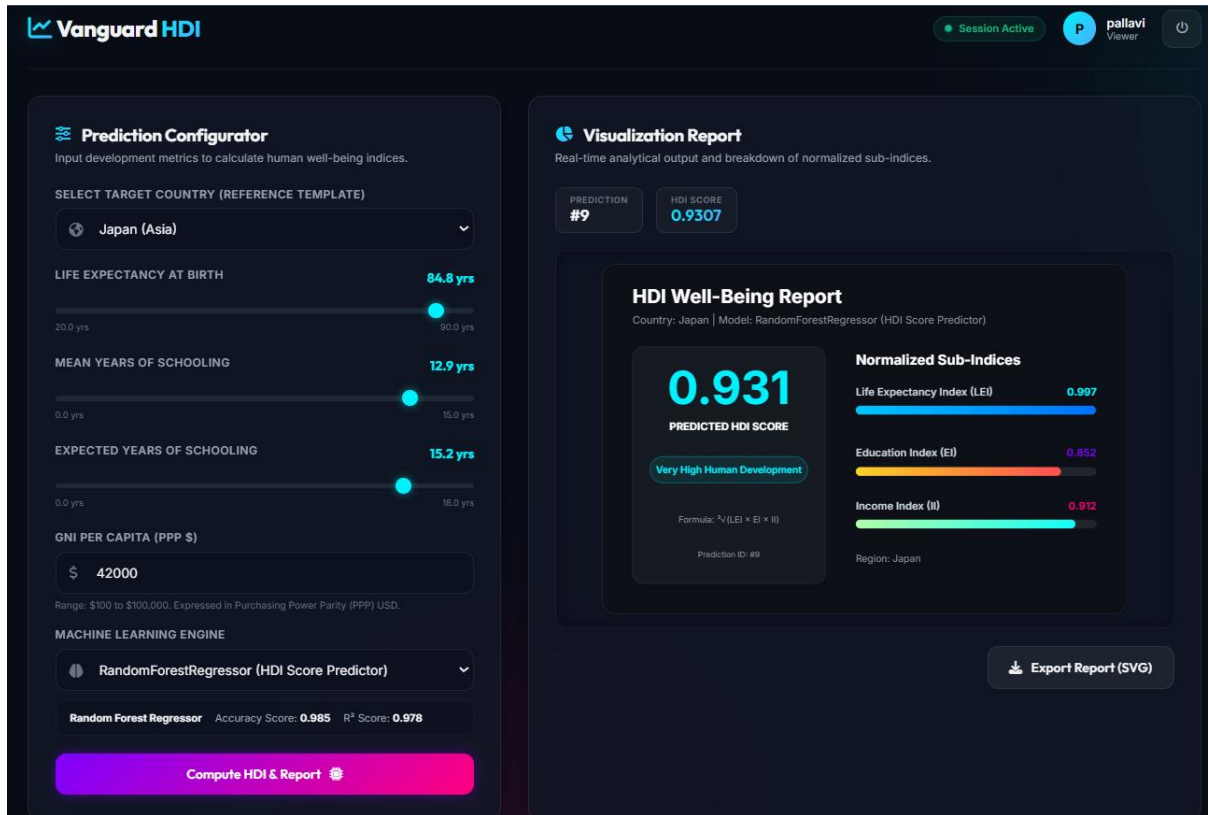
Real-time analytical output and breakdown of normalized sub-indices.



Analytical Model Ready

Configure and run the prediction model to generate interactive SVG graphs and scores.

Prediction Results



History

Historical Prediction Logs

10 Records

Timestamp	Analyst	Country/Region	ML Engine	Expected Metrics	Predicted HDI	Classification	Action
2026-07-14 15:47:44	pallavi Viewer	Niger Africa	Random Forest Regressor RandomForestRegressor (HDI Score Predictor)	📊 Live-Ex 📄 Edu-Idx	0.2394	Low Human Development	🔍 View
2026-07-14 15:47:21	pallavi Viewer	Japan Asia	Random Forest Regressor RandomForestRegressor (HDI Score Predictor)	📊 Live-Ex 📄 Edu-Idx	0.9307	Very High Human Development	🔍 View
2026-07-14 15:40:23	prameela Viewer	Niger Africa	Random Forest Regressor RandomForestRegressor (HDI Score Predictor)	📊 Live-Ex 📄 Edu-Idx	0.4358	Low Human Development	🔍 View
2026-07-14 15:40:09	prameela Viewer	India Asia	Random Forest Regressor RandomForestRegressor (HDI Score Predictor)	📊 Live-Ex 📄 Edu-Idx	0.7576	High Human Development	🔍 View
2026-07-14 15:39:46	prameela Viewer	India Asia	Random Forest Regressor RandomForestRegressor (HDI Score Predictor)	📊 Live-Ex 📄 Edu-Idx	0.6425	Medium Human Development	🔍 View
2026-07-13 16:47:49	prameela Viewer	Australia Oceania	Random Forest Regressor RandomForestRegressor (HDI Score Predictor)	📊 Live-Ex 📄 Edu-Idx	0.9425	Very High Human Development	🔍 View
2026-07-07 15:14:39	Manoj Analyst	India Asia	Random Forest Regressor RandomForestRegressor (HDI Score Predictor)	📊 Live-Ex 📄 Edu-Idx	0.6425	Medium Human Development	🔍 View
2026-07-02 23:36:20	Hardik Analyst	India Asia	Linear Regression LinearRegression (Baseline HDI)	📊 Live-Ex 📄 Edu-Idx	0.6425	Medium Human Development	🔍 View
2026-06-30 08:05:10	Alice Smith Analyst	Switzerland Europe	Random Forest Regressor RandomForestRegressor (HDI Score Predictor)	📊 Live-Ex 📄 Edu-Idx	0.9879	Very High Human Development	🔍 View

8. ADVANTAGES & DISADVANTAGES

Advantages

- Provides fast and accurate Human Development Index (HDI) prediction.
- Reduces manual calculations and human errors.
- Uses Machine Learning for intelligent prediction.
- Easy-to-use web interface for users.
- Stores prediction history in the SQLite database.
- Generates prediction reports for future reference.
- Helps governments, researchers, and educational institutions analyze development indicators.
- Saves time by providing instant prediction results.
- Modular architecture makes the system easy to maintain and upgrade.
- Can be extended to support multiple machine learning algorithms and real-time datasets.

Disadvantages

- Prediction accuracy depends on the quality and size of the training dataset.
- Uses a Linear Regression model, which may not capture complex non-linear relationships.
- The model requires retraining when new HDI data becomes available.
- Currently predicts HDI using limited input parameters only.
- SQLite is suitable for small-scale applications but not ideal for large enterprise deployments.
- Internet or local server access is required to use the web application.

9. CONCLUSION

The A Comprehensive Measure of Well-Being – Human Development Index (HDI) Prediction System successfully predicts the Human Development Index using Machine Learning techniques. The Linear Regression model was trained using historical HDI data and integrated into a Flask-based web application to provide real-time predictions based on key development indicators such as Life Expectancy, Expected Years of Schooling, Mean Years of Schooling, and Gross National Income (GNI) per Capita.

The system simplifies the process of HDI prediction by automating data preprocessing, prediction, and result generation. It also stores prediction history in a SQLite database and provides an easy-to-use web interface for users. The application reduces manual effort, minimizes calculation errors, and delivers fast and reliable predictions.

Overall, the project demonstrates the practical use of Machine Learning and web technologies in socio-economic analysis. It can serve as a valuable tool for researchers, students, government organizations, and policy makers to understand and analyze human development more efficiently. The project meets its objectives by providing an accurate, user-friendly, and scalable solution for HDI prediction and analysis.

10. FUTURE SCOPE

- Integrate advanced Machine Learning algorithms such as Random Forest, XGBoost, and Neural Networks to improve prediction accuracy.
- Use real-time development data from organizations such as the United Nations Development Programme (UNDP) and the World Bank.
- Develop an interactive dashboard with graphical visualization and country-wise comparison.
- Add year-wise HDI trend analysis and future HDI forecasting.
- Implement Explainable Artificial Intelligence (XAI) to provide reasons behind each prediction.
- Develop a mobile application for easy access to HDI predictions.
- Deploy the application on cloud platforms such as AWS, Microsoft Azure, or Google Cloud Platform.
- Support multiple countries and regional-level HDI analysis with interactive maps.
- Integrate data visualization tools for better analysis of health, education, and income indicators.
- Expand the system to provide policy recommendations based on predicted HDI values.

11. APPENDIX

11.1 Source Code

App.py

```
import os
import math
import datetime
```

```

import pickle
import numpy as np
import sys
from flask import Flask, request, jsonify, render_template, send_from_directory
from contextlib import contextmanager

# Ensure models module can be imported if script is run directly or from Flask
sys.path.append(os.path.dirname(__file__))

from models import (
    SessionLocal, User, Country, Session as DBSession, Dataset, MLModel,
    HDIInputData, HDIPrediction, VisualizationReport
)

app = Flask(__name__, template_folder="templates", static_folder="static")

# Ensure static directories exist
STATIC_REPORTS_DIR = os.path.join(os.path.dirname(__file__), "static", "reports")
os.makedirs(STATIC_REPORTS_DIR, exist_ok=True)

# Load the machine learning model trained in scikit-learn
MODEL_PATH = os.path.join(os.path.dirname(__file__), "HDI.pkl")
ml_model_pickle = None

def load_ml_model():
    global ml_model_pickle
    if os.path.exists(MODEL_PATH):
        try:
            with open(MODEL_PATH, "rb") as f:
                ml_model_pickle = pickle.load(f)
            print("Successfully loaded scikit-learn ML model from HDI.pkl")
        except Exception as e:
            print(f"Error loading ML model: {e}")
    else:
        print(f"Warning: ML model not found at {MODEL_PATH}. Prediction falls back to analytical formula.")

# Database helper context manager
@contextmanager
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

# Analytical breakdown calculation (LEI, EI, II)
def calculate_hdi_breakdown(life_expectancy, mean_years_schooling,
    expected_years_schooling, gnl_per_capita):
    # Normalized Life Expectancy Index
    lei = (life_expectancy - 20.0) / 65.0

```

```

lei = max(0.0, min(1.0, lei))

# Normalized Education Index
mysi = mean_years_schooling / 15.0
mysi = max(0.0, min(1.0, mysi))

eysi = expected_years_schooling / 18.0
eysi = max(0.0, min(1.0, eysi))

ei = (mysi + eysi) / 2.0

# Normalized Income Index
gni_val = max(100.0, gnl_per_capita)
ii = (math.log(gni_val) - math.log(100.0)) / (math.log(75000.0) - math.log(100.0))
ii = max(0.0, min(1.0, ii))

return lei, ei, ii

# Generate SVG Report
def generate_svg_report(prediction_id, country_name, model_name, hdi_score, category, lei,
ei, ii):
    if hdi_score >= 0.800:
        hdi_color = "#00F2FE"
    elif hdi_score >= 0.700:
        hdi_color = "#4FACFE"
    elif hdi_score >= 0.550:
        hdi_color = "#FAD961"
    else:
        hdi_color = "#F76B1C"

    lei_pct = lei * 100
    ei_pct = ei * 100
    ii_pct = ii * 100

    svg = f'<svg width="600" height="400" viewBox="0 0 600 400" fill="none"
xmlns="http://www.w3.org/2000/svg">
        <rect width="600" height="400" rx="16" fill="#0D1117" />
        <rect x="0.5" y="0.5" width="599" height="399" rx="15.5" stroke="#30363D" stroke-
opacity="0.5" />

        <text x="35" y="45" font-family="Inter', system-ui, sans-serif" font-weight="800" font-
size="22" fill="#FFFFFF">HDI Well-Being Report</text>
        <text x="35" y="68" font-family="Inter', system-ui, sans-serif" font-weight="400" font-
size="12" fill="#8B949E">Country: {country_name} | Model: {model_name}</text>

        <rect x="35" y="95" width="220" height="270" rx="12" fill="#161B22"
stroke="#21262D" />
        <text x="145" y="160" text-anchor="middle" font-family="Inter', system-ui, sans-serif"
font-weight="800" font-size="52" fill="{hdi_color}">{hdi_score:.3f}</text>
        <text x="145" y="190" text-anchor="middle" font-family="Inter', system-ui, sans-serif"
font-weight="700" font-size="12" fill="#FFFFFF">PREDICTED HDI SCORE</text>

```



```

<rect x="55" y="220" width="180" height="30" rx="15" fill="{hdi_color}15"
stroke="{hdi_color}" stroke-opacity="0.3" />
<text x="145" y="239" text-anchor="middle" font-family="'Inter', system-ui, sans-serif"
font-weight="600" font-size="11" fill="{hdi_color}">{category}</text>

<text x="145" y="295" text-anchor="middle" font-family="'Inter', system-ui, sans-serif"
font-weight="400" font-size="10" fill="#8B949E">Formula:  $\sqrt[3]{(LEI \times EI \times II)}$ </text>
<text x="145" y="335" text-anchor="middle" font-family="'Inter', system-ui, sans-serif"
font-weight="400" font-size="9" fill="#8B949E">Prediction ID: #{prediction_id}</text>

<text x="290" y="115" font-family="'Inter', system-ui, sans-serif" font-weight="700"
font-size="16" fill="#FFFFFF">Normalized Sub-Indices</text>

<text x="290" y="150" font-family="'Inter', system-ui, sans-serif" font-weight="600"
font-size="12" fill="#C9D1D9">Life Expectancy Index (LEI)</text>
<text x="565" y="150" text-anchor="end" font-family="'Inter', system-ui, sans-serif"
font-weight="700" font-size="12" fill="#00F2FE">{lei:.3f}</text>
<rect x="290" y="162" width="275" height="10" rx="5" fill="#21262D" />
<rect x="290" y="162" width="{lei_pct * 2.75:.1f}" height="10" rx="5"
fill="url(#lei_grad)" />

<text x="290" y="220" font-family="'Inter', system-ui, sans-serif" font-weight="600"
font-size="12" fill="#C9D1D9">Education Index (EI)</text>
<text x="565" y="220" text-anchor="end" font-family="'Inter', system-ui, sans-serif"
font-weight="700" font-size="12" fill="#7F00FF">{ei:.3f}</text>
<rect x="290" y="232" width="275" height="10" rx="5" fill="#21262D" />
<rect x="290" y="232" width="{ei_pct * 2.75:.1f}" height="10" rx="5"
fill="url(#ei_grad)" />

<text x="290" y="280" font-family="'Inter', system-ui, sans-serif" font-weight="600"
font-size="12" fill="#C9D1D9">Income Index (II)</text>
<text x="565" y="280" text-anchor="end" font-family="'Inter', system-ui, sans-serif"
font-weight="700" font-size="12" fill="#FF007F">{ii:.3f}</text>
<rect x="290" y="292" width="275" height="10" rx="5" fill="#21262D" />
<rect x="290" y="292" width="{ii_pct * 2.75:.1f}" height="10" rx="5"
fill="url(#ii_grad)" />

<text x="290" y="340" font-family="'Inter', system-ui, sans-serif" font-weight="400"
font-size="11" fill="#8B949E">Region: {country_name}</text>

<defs>
<linearGradient id="lei_grad" x1="0" y1="0" x2="1" y2="0">
<stop offset="0%" stop-color="#00C6FF" />
<stop offset="100%" stop-color="#0072FF" />
</linearGradient>
<linearGradient id="ei_grad" x1="0" y1="0" x2="1" y2="0">
<stop offset="0%" stop-color="#F9D423" />
<stop offset="100%" stop-color="#FF4E50" />
</linearGradient>
<linearGradient id="ii_grad" x1="0" y1="0" x2="1" y2="0">

```

```

        <stop offset="0%" stop-color="#B3FFAB" />
        <stop offset="100%" stop-color="#12FFF7" />
    </linearGradient>
</defs>
</svg>""
return svg

```

HTTP Frontend Routes

```
@app.route("/")
```

```
def home():
```

```
    return render_template("home.html")
```

```
@app.route("/dashboard")
```

```
def dashboard():
```

```
    return render_template("index.html")
```

Traditional HTML Form POST Predict Route

```
@app.route("/predict", methods=["POST"])
```

```
def predict():
```

```
    try:
```

```
        # Retrieve input values from traditional form submission
```

```
        life_expectancy = float(request.form.get("life_expectancy", 70.0))
```

```
        mean_schooling = float(request.form.get("mean_years_schooling", 7.0))
```

```
        expected_schooling = float(request.form.get("expected_years_schooling", 10.0))
```

```
        gni = float(request.form.get("gnl_per_capita", 15000.0))
```

```
        score = None
```

```
        if ml_model_pickle is not None:
```

```
            try:
```

```
                features = np.array([[life_expectancy, mean_schooling, expected_schooling, gni]])
```

```
                score = float(ml_model_pickle.predict(features)[0])
```

```
            except Exception as e:
```

```
                print(f'Error predicting with pickled model: {e}')

```

```
        if score is None:
```

```
            lei, ei, ii = calculate_hdi_breakdown(life_expectancy, mean_schooling,
            expected_schooling, gni)
```

```
            score = (lei * ei * ii) ** (1.0 / 3.0)
```

```
        score = max(0.0, min(1.0, score))
```

```
        rounded_score = round(score, 4)
```

```
        return render_template(
```

```
            "predict.html",
```

```
            score=rounded_score,
```

```
            life_expectancy=life_expectancy,
```

```
            mean_schooling=mean_schooling,
```

```
            expected_schooling=expected_schooling,
```

```
            gni=gni
```

```
        )
```

```
    except Exception as e:
```

```
return f"Error during prediction calculation: {e}", 400
```

```
# API Routes
```

```
@app.route("/api/auth/login", methods=["POST"])
```

```
def login():
```

```
    req = request.get_json()
```

```
    name = req.get("name")
```

```
    email = req.get("email")
```

```
    role = req.get("role")
```

```
if not name or not email or not role:
```

```
    return jsonify({"detail": "Missing parameters"}), 400
```

```
with get_db() as db:
```

```
    user = db.query(User).filter(User.email == email).first()
```

```
    if not user:
```

```
        user = User(name=name, email=email, role=role)
```

```
        db.add(user)
```

```
        db.commit()
```

```
        db.refresh(user)
```

```
session = DBSession(
```

```
    user_id=user.user_id,
```

```
    login_time=datetime.datetime.utcnow(),
```

```
    status="Active"
```

```
)
```

```
db.add(session)
```

```
db.commit()
```

```
db.refresh(session)
```

```
return jsonify({
```

```
    "user_id": user.user_id,
```

```
    "name": user.name,
```

```
    "email": user.email,
```

```
    "role": user.role,
```

```
    "session_id": session.session_id
```

```
})
```

```
@app.route("/api/auth/logout/<int:session_id>", methods=["POST"])
```

```
def logout(session_id):
```

```
    with get_db() as db:
```

```
        session = db.query(DBSession).filter(DBSession.session_id == session_id).first()
```

```
        if not session:
```

```
            return jsonify({"detail": "Session not found"}), 404
```

```
        session.logout_time = datetime.datetime.utcnow()
```

```
        session.status = "Completed"
```

```
        db.commit()
```

```
        return jsonify({"message": "Logged out successfully"})
```

```
@app.route("/api/countries", methods=["GET"])
```

```

def get_countries():
    with get_db() as db:
        countries = db.query(Country).all()
        return jsonify([ {
            "country_id": c.country_id,
            "country_name": c.country_name,
            "region": c.region,
            "population": c.population
        } for c in countries])

@app.route("/api/datasets", methods=["GET"])
def get_datasets():
    with get_db() as db:
        datasets = db.query(Dataset).all()
        return jsonify([ {
            "dataset_id": d.dataset_id,
            "dataset_name": d.dataset_name,
            "source": d.source,
            "total_rows": d.total_rows,
            "total_columns": d.total_columns
        } for d in datasets])

@app.route("/api/models", methods=["GET"])
def get_models():
    with get_db() as db:
        models = db.query(MLModel).all()
        return jsonify([ {
            "model_id": m.model_id,
            "dataset_id": m.dataset_id,
            "model_name": m.model_name,
            "algorithm_used": m.algorithm_used,
            "accuracy_score": m.accuracy_score,
            "r2_score": m.r2_score,
            "model_file_path": m.model_file_path
        } for m in models])

@app.route("/api/predict", methods=["POST"])
def predict_hdi():
    req = request.get_json()
    user_id = req.get("user_id")
    country_id = req.get("country_id")
    life_expectancy = req.get("life_expectancy")
    mean_years_schooling = req.get("mean_years_schooling")
    expected_years_schooling = req.get("expected_years_schooling")
    gnl_per_capita = req.get("gnl_per_capita")
    model_id = req.get("model_id")

    with get_db() as db:
        # Validate entities
        user = db.query(User).filter(User.user_id == user_id).first()
        if not user:

```

```

    return jsonify({"detail": "User not found"}), 400

country = db.query(Country).filter(Country.country_id == country_id).first()
if not country:
    return jsonify({"detail": "Country not found"}), 400

ml_model = db.query(MLModel).filter(MLModel.model_id == model_id).first()
if not ml_model:
    return jsonify({"detail": "Model not found"}), 400

# Run ML model prediction or analytical fallback
score = None
if ml_model_pickle is not None:
    try:
        features = np.array([[life_expectancy, mean_years_schooling,
expected_years_schooling, gnl_per_capita]])
        score = float(ml_model_pickle.predict(features)[0])
    except Exception as e:
        print(f'Error predicting with pickled model: {e}')

if score is None:
    # Fallback to analytical calculation if pickle fails or doesn't exist
    # Geometric mean formula with a minor variance based on selected model
    lei, ei, ii = calculate_hdi_breakdown(life_expectancy, mean_years_schooling,
expected_years_schooling, gnl_per_capita)
    base_score = (lei * ei * ii) ** (1.0 / 3.0)

    # Model variance to show some variation
    model_variance = 0.0
    if "neural" in ml_model.model_name.lower():
        model_variance = -0.012
    elif "linear" in ml_model.model_name.lower():
        model_variance = 0.008
    elif "forest" in ml_model.model_name.lower():
        model_variance = -0.002

    score = max(0.0, min(1.0, base_score + model_variance))

# Determine classification category
if score >= 0.800:
    category = "Very High Human Development"
elif score >= 0.700:
    category = "High Human Development"
elif score >= 0.550:
    category = "Medium Human Development"
else:
    category = "Low Human Development"

# Save input record
input_data = HDIInputData(
    user_id=user_id,

```

```

        country_id=country_id,
        life_expectancy=life_expectancy,
        mean_years_schooling=mean_years_schooling,
        expected_years_schooling=expected_years_schooling,
        gnl_per_capita=gnl_per_capita
    )
    db.add(input_data)
    db.commit()
    db.refresh(input_data)

    # Save prediction record
    prediction = HDIPrediction(
        input_id=input_data.input_id,
        model_id=model_id,
        predicted_hdi_score=score,
        hdi_category=category
    )
    db.add(prediction)
    db.commit()
    db.refresh(prediction)

    # Generate SVG report
    lei, ei, ii = calculate_hdi_breakdown(life_expectancy, mean_years_schooling,
    expected_years_schooling, gnl_per_capita)
    svg_content = generate_svg_report(
        prediction.prediction_id,
        country.country_name,
        ml_model.model_name,
        score,
        category,
        lei,
        ei,
        ii
    )

    # Write SVG file to Flask/static/reports/
    rel_graph_path = f"static/reports/prediction_{prediction.prediction_id}.svg"
    abs_graph_path = os.path.join(os.path.dirname(__file__), rel_graph_path)
    with open(abs_graph_path, "w", encoding="utf-8") as f:
        f.write(svg_content)

    # Save visualization report record
    report = VisualizationReport(
        prediction_id=prediction.prediction_id,
        graph_path=rel_graph_path,
        report_type="SVG Bar & Gauge Breakdown"
    )
    db.add(report)
    db.commit()
    db.refresh(report)

```

```

return jsonify({
    "prediction_id": prediction.prediction_id,
    "score": score,
    "category": category,
    "report_id": report.report_id,
    "graph_path": f'/{rel_graph_path}'
})

@app.route("/api/predictions", methods=["GET"])
def get_predictions():
    with get_db() as db:
        results = db.query(HDIPrediction).order_by(HDIPrediction.prediction_time.desc()).all()
        out = []
        for r in results:
            inp = r.input_data
            user = inp.user
            country = inp.country
            model = r.model
            report = db.query(VisualizationReport).filter(VisualizationReport.prediction_id ==
r.prediction_id).first()

            out.append({
                "prediction_id": r.prediction_id,
                "predicted_hdi_score": round(r.predicted_hdi_score, 4),
                "hdi_category": r.hdi_category,
                "prediction_time": r.prediction_time.strftime("%Y-%m-%d %H:%M:%S"),
                "user": {"name": user.name, "role": user.role},
                "country": {"name": country.country_name, "region": country.region},
                "model": {"name": model.model_name, "algorithm": model.algorithm_used},
                "report_path": f'/{report.graph_path}' if report else None
            })
        return jsonify(out)

# Initialize Flask ML model loading on startup
load_ml_model()

if __name__ == "__main__":
    app.run(host="127.0.0.1", port=8000, debug=True)

```

Models.py

```

import datetime
from sqlalchemy import (
    Column, Integer, String, Float, DateTime, ForeignKey, Text, create_engine
)
from sqlalchemy.orm import relationship, declarative_base, sessionmaker

Base = declarative_base()

class Country(Base):
    __tablename__ = 'country'

```

```

country_id = Column(Integer, primary_key=True, autoincrement=True)
country_name = Column(String(255), nullable=False)
region = Column(String(100), nullable=False)
population = Column(Integer)
created_at = Column(DateTime, default=datetime.datetime.utcnow)

# Relationships
inputs = relationship("HDIInputData", back_populates="country", cascade="all, delete-orphan")

class User(Base):
    __tablename__ = 'user'

    user_id = Column(Integer, primary_key=True, autoincrement=True)
    name = Column(String(255), nullable=False)
    email = Column(String(255), unique=True, nullable=False)
    role = Column(String(50), nullable=False)
    created_at = Column(DateTime, default=datetime.datetime.utcnow)

    # Relationships
    sessions = relationship("Session", back_populates="user", cascade="all, delete-orphan")
    inputs = relationship("HDIInputData", back_populates="user", cascade="all, delete-orphan")

class Session(Base):
    __tablename__ = 'session'

    session_id = Column(Integer, primary_key=True, autoincrement=True)
    user_id = Column(Integer, ForeignKey('user.user_id', ondelete='CASCADE'),
    nullable=False)
    login_time = Column(DateTime, default=datetime.datetime.utcnow, nullable=False)
    logout_time = Column(DateTime)
    status = Column(String(20), nullable=False, default='Active')

    # Relationships
    user = relationship("User", back_populates="sessions")

class Dataset(Base):
    __tablename__ = 'dataset'

    dataset_id = Column(Integer, primary_key=True, autoincrement=True)
    dataset_name = Column(String(255), nullable=False)
    source = Column(String(255), nullable=False)
    total_rows = Column(Integer)
    total_columns = Column(Integer)
    uploaded_at = Column(DateTime, default=datetime.datetime.utcnow)

    # Relationships
    models = relationship("MLModel", back_populates="dataset")

```



```

class MLModel(Base):
    __tablename__ = 'ml_model'

    model_id = Column(Integer, primary_key=True, autoincrement=True)
    dataset_id = Column(Integer, ForeignKey('dataset.dataset_id', ondelete='SET NULL'),
nullable=False)
    model_name = Column(String(255), nullable=False)
    algorithm_used = Column(String(100), nullable=False)
    accuracy_score = Column(Float)
    r2_score = Column(Float)
    model_file_path = Column(String(255))
    trained_at = Column(DateTime, default=datetime.datetime.utcnow)

    # Relationships
    dataset = relationship("Dataset", back_populates="models")
    predictions = relationship("HDIPrediction", back_populates="model", cascade="all,
delete-orphan")

class HDIInputData(Base):
    __tablename__ = 'hdi_input_data'

    input_id = Column(Integer, primary_key=True, autoincrement=True)
    user_id = Column(Integer, ForeignKey('user.user_id', ondelete='CASCADE'),
nullable=False)
    country_id = Column(Integer, ForeignKey('country.country_id', ondelete='CASCADE'),
nullable=False)
    life_expectancy = Column(Float, nullable=False)
    mean_years_schooling = Column(Float, nullable=False)
    expected_years_schooling = Column(Float, nullable=False)
    gnl_per_capita = Column(Float, nullable=False) # matching ER diagram 'gnl_per_capita'
    created_at = Column(DateTime, default=datetime.datetime.utcnow)

    # Relationships
    user = relationship("User", back_populates="inputs")
    country = relationship("Country", back_populates="inputs")
    prediction = relationship("HDIPrediction", back_populates="input_data", uselist=False,
cascade="all, delete-orphan")

class HDIPrediction(Base):
    __tablename__ = 'hdi_prediction'

    prediction_id = Column(Integer, primary_key=True, autoincrement=True)
    input_id = Column(Integer, ForeignKey('hdi_input_data.input_id', ondelete='CASCADE'),
unique=True, nullable=False)
    model_id = Column(Integer, ForeignKey('ml_model.model_id', ondelete='CASCADE'),
nullable=False)
    predicted_hdi_score = Column(Float, nullable=False)
    hdi_category = Column(String(50), nullable=False)
    prediction_time = Column(DateTime, default=datetime.datetime.utcnow)

    # Relationships

```

```

input_data = relationship("HDIInputData", back_populates="prediction")
model = relationship("MLModel", back_populates="predictions")
reports = relationship("VisualizationReport", back_populates="prediction", cascade="all,
delete-orphan")

```

```

class VisualizationReport(Base):
    __tablename__ = 'visualization_report'

    report_id = Column(Integer, primary_key=True, autoincrement=True)
    prediction_id = Column(Integer, ForeignKey('hdi_prediction.prediction_id',
ondelete='CASCADE'), nullable=False)
    graph_path = Column(String(255), nullable=False)
    report_type = Column(String(50), nullable=False)
    generated_at = Column(DateTime, default=datetime.datetime.utcnow)

    # Relationships
    prediction = relationship("HDIPrediction", back_populates="reports")

# Database Connection setup helper
DATABASE_URL = "sqlite:///./hdi_wellbeing.db"
engine = create_engine(DATABASE_URL, connect_args={"check_same_thread": False})
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

def init_db():
    Base.metadata.create_all(bind=engine)

```

Index.html

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Vanguard HDI Analyzer - Global Well-Being & Human Development
Predictor</title>
    <meta name="description" content="An advanced predictive analytics platform for
Human Development Index (HDI) forecasting, powered by machine learning and historical
country data.">
    <!-- Google Fonts -->
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link
href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700&family=
Outfit:wght@400;600;700;800&display=swap" rel="stylesheet">
    <!-- FontAwesome for Premium Icons -->
    <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/font-
awesome/6.4.0/css/all.min.css">
    <!-- Custom Style Sheet -->
    <link rel="stylesheet" href="/static/index.css">
</head>
<body>

```

```

<!-- Background Elements -->
<div class="glow-sphere sphere-1"></div>
<div class="glow-sphere sphere-2"></div>
<div class="glow-sphere sphere-3"></div>

<!-- Login Overlay -->
<div id="login-modal" class="modal-overlay">
  <div class="login-card">
    <div class="brand-logo">
      <i class="fa-solid fa-chart-line logo-icon"></i>
      <span class="logo-text">Vanguard <span class="accent-
text">HDI</span></span>
    </div>
    <h2>Access Predictive Workspace</h2>
    <p>Enter your details to initiate an analysis session.</p>

    <form id="login-form">
      <div class="form-group">
        <label for="login-name">Full Name</label>
        <div class="input-wrapper">
          <i class="fa-regular fa-user input-icon"></i>
          <input type="text" id="login-name" required placeholder="e.g. Alice Smith">
        </div>
      </div>
      <div class="form-group">
        <label for="login-email">Work Email</label>
        <div class="input-wrapper">
          <i class="fa-regular fa-envelope input-icon"></i>
          <input type="email" id="login-email" required
placeholder="name@wellbeing.org">
        </div>
      </div>
      <div class="form-group">
        <label for="login-role">Professional Role</label>
        <div class="input-wrapper">
          <i class="fa-solid fa-briefcase input-icon"></i>
          <select id="login-role">
            <option value="Analyst">Global Policy Analyst</option>
            <option value="Data Scientist">Senior Data Scientist</option>
            <option value="Viewer">Guest Observer</option>
          </select>
        </div>
      </div>
      <button type="submit" class="btn btn-primary btn-block" id="login-submit-btn">
        <span>Initialize Session</span>
        <i class="fa-solid fa-right-to-bracket"></i>
      </button>
    </form>
  </div>
</div>

```

```

<!-- Main Dashboard Application (Hidden initially) -->
<div id="app-container" class="app-layout hidden">
  <!-- Top Navigation -->
  <header class="main-header">
    <div class="brand">
      <i class="fa-solid fa-chart-line logo-icon"></i>
      <span class="logo-text">Vanguard <span class="accent-
text">HDI</span></span>
    </div>

    <div class="header-right">
      <div class="session-badge">
        <span class="pulse-indicator"></span>
        <span class="session-text">Session Active</span>
      </div>

      <div class="user-profile">
        <div class="user-avatar" id="user-avatar-text">U</div>
        <div class="user-meta">
          <span class="user-name" id="display-user-name">User Name</span>
          <span class="user-role" id="display-user-role">Role</span>
        </div>
      </div>

      <button id="logout-btn" class="btn-icon" title="End Session">
        <i class="fa-solid fa-power-off"></i>
      </button>
    </div>
  </header>

  <!-- Dashboard Body Grid -->
  <main class="dashboard-grid">
    <!-- Left Panel: Form Input & Configuration -->
    <section class="dashboard-card card-input-panel">
      <div class="card-header">
        <div class="header-title">
          <i class="fa-solid fa-sliders icon-accent"></i>
          <h2>Prediction Configurator</h2>
        </div>
        <p class="subtitle">Input development metrics to calculate human well-being
indices.</p>
      </div>

      <form id="prediction-form" class="prediction-form" action="/predict"
method="POST">
        <!-- Country Pre-population Selector -->
        <div class="form-group">
          <label for="predict-country">Select Target Country (Reference
template)</label>
          <div class="input-wrapper">
            <i class="fa-solid fa-earth-americas input-icon"></i>

```

```

        <select id="predict-country" name="country_id" required>
            <option value="">Select country...</option>
            <!-- Will be filled dynamically -->
        </select>
    </div>
</div>

<!-- Life Expectancy Metric -->
<div class="form-group slider-group">
    <div class="label-row">
        <label for="life-expectancy">Life Expectancy at Birth</label>
        <span class="metric-value"><span id="val-life">70.0</span> yrs</span>
    </div>
    <div class="range-wrapper">
        <input type="range" id="life-expectancy" name="life_expectancy"
min="20" max="90" step="0.1" value="70">
        <div class="range-labels">
            <span>20.0 yrs</span>
            <span>90.0 yrs</span>
        </div>
    </div>
</div>

<!-- Mean Years of Schooling -->
<div class="form-group slider-group">
    <div class="label-row">
        <label for="mean-schooling">Mean Years of Schooling</label>
        <span class="metric-value"><span id="val-mean-school">7.0</span>
yrs</span>
    </div>
    <div class="range-wrapper">
        <input type="range" id="mean-schooling" name="mean_years_schooling"
min="0" max="15" step="0.1" value="7">
        <div class="range-labels">
            <span>0.0 yrs</span>
            <span>15.0 yrs</span>
        </div>
    </div>
</div>

<!-- Expected Years of Schooling -->
<div class="form-group slider-group">
    <div class="label-row">
        <label for="expected-schooling">Expected Years of Schooling</label>
        <span class="metric-value"><span id="val-exp-school">10.0</span>
yrs</span>
    </div>
    <div class="range-wrapper">
        <input type="range" id="expected-schooling"
name="expected_years_schooling" min="0" max="18" step="0.1" value="10">
        <div class="range-labels">

```

```

        <span>0.0 yrs</span>
        <span>18.0 yrs</span>
    </div>
</div>
</div>

<!-- GNI per Capita -->
<div class="form-group">
    <label for="gnl-per-capita">GNI per Capita (PPP $)</label>
    <div class="input-wrapper">
        <i class="fa-solid fa-dollar-sign input-icon"></i>
        <input type="number" id="gnl-per-capita" name="gnl_per_capita"
min="100" max="100000" step="100" value="15000" required>
    </div>
    <p class="input-tip">Range: $100 to $100,000. Expressed in Purchasing
Power Parity (PPP) USD.</p>
</div>

<!-- Machine Learning Model -->
<div class="form-group">
    <label for="predict-model">Machine Learning Engine</label>
    <div class="input-wrapper">
        <i class="fa-solid fa-brain input-icon"></i>
        <select id="predict-model" name="model_id" required>
            <!-- Will be filled dynamically -->
        </select>
    </div>
    <!-- Model Details Sub-panel -->
    <div id="model-details-card" class="model-details-card hidden">
        <span class="model-meta-item"><strong id="model-algo-
name">N/A</strong></span>
        <span class="model-meta-item">Accuracy Score: <strong id="model-
accuracy">N/A</strong></span>
        <span class="model-meta-item">R2 Score: <strong id="model-
r2">N/A</strong></span>
    </div>
</div>

    <button type="submit" class="btn btn-accent btn-block" id="predict-submit-
btn">
        <span>Compute HDI & Report</span>
        <i class="fa-solid fa-microchip"></i>
    </button>
</form>
</section>

<!-- Right Panel: Prediction Results & Visualizations -->
<section class="dashboard-card card-output-panel">
    <div class="card-header">
        <div class="header-title">
            <i class="fa-solid fa-chart-pie icon-accent"></i>

```

```

        <h2>Visualization Report</h2>
    </div>
    <p class="subtitle">Real-time analytical output and breakdown of normalized
sub-indices.</p>
</div>

<div class="output-content">
    <!-- Blank state when no prediction runs yet -->
    <div id="output-blank-state" class="output-blank-state">
        <div class="blank-icon-wrapper">
            <i class="fa-solid fa-chart-line-up blank-icon"></i>
        </div>
        <h3>Analytical Model Ready</h3>
        <p>Configure and run the prediction model to generate interactive SVG
graphs and scores.</p>
    </div>

    <!-- Loader -->
    <div id="output-loader" class="output-loader hidden">
        <div class="spinner"></div>
        <p>Running ML Model Predictions...</p>
    </div>

    <!-- Active Report Section -->
    <div id="active-report-section" class="active-report-section hidden">
        <div class="report-meta-badges">
            <div class="report-badge-item">
                <span class="badge-label">PREDICTION</span>
                <span class="badge-val" id="report-pred-id">#1</span>
            </div>
            <div class="report-badge-item">
                <span class="badge-label">HDI SCORE</span>
                <span class="badge-val text-accent" id="report-pred-
score">0.000</span>
            </div>
        </div>

        <!-- Embedded SVG graph container -->
        <div id="svg-report-container" class="svg-report-container">
            <!-- SVG element will be appended here -->
        </div>

        <div class="action-footer">
            <button id="download-report-btn" class="btn btn-secondary">
                <i class="fa-solid fa-download"></i>
                <span>Export Report (SVG)</span>
            </button>
        </div>
    </div>
</div>
</section>

```

```

<!-- Bottom Panel: Historical Prediction Logs Table -->
<section class="dashboard-card card-logs-panel">
  <div class="card-header logs-header">
    <div class="header-title">
      <i class="fa-solid fa-list-check icon-accent"></i>
      <h2>Historical Prediction Logs</h2>
    </div>
    <span class="badge-counter" id="logs-count">0 Records</span>
  </div>

  <div class="table-scroll-wrapper">
    <table class="logs-table" id="logs-table">
      <thead>
        <tr>
          <th>Timestamp</th>
          <th>Analyst</th>
          <th>Country/Region</th>
          <th>ML Engine</th>
          <th>Expected Metrics</th>
          <th>Predicted HDI</th>
          <th>Classification</th>
          <th>Action</th>
        </tr>
      </thead>
      <tbody id="logs-table-body">
        <!-- Rows filled dynamically -->
      </tbody>
    </table>

    <div id="table-blank-state" class="table-blank-state">
      <p>No historical analysis records available in current database.</p>
    </div>
  </div>
</section>
</main>
</div>
<script src="/static/index.js"></script>
</body>
</html>

```

Home.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Vanguard HDI - Human Development Index Forecasting System</title>
  <meta name="description" content="An advanced predictive analytics platform for
Human Development Index (HDI) forecasting, powered by machine learning and historical
country data.">

```



```
<!-- Google Fonts -->
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link
href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700&family=
Outfit:wght@400;600;700;800&display=swap" rel="stylesheet">
<!-- FontAwesome for Premium Icons -->
<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/@fortawesome/fontawesome-free@6.4.0/css/all.min.css">
<!-- Custom Style Sheet -->
<link rel="stylesheet" href="/static/index.css">
<style>
  .home-container {
    display: flex;
    flex-direction: column;
    align-items: center;
    justify-content: center;
    min-height: 100vh;
    padding: 2rem;
    max-width: 900px;
    margin: 0 auto;
    text-align: center;
    position: relative;
    z-index: 10;
  }
  .intro-card {
    background: var(--card-bg);
    border: 1px solid var(--card-border);
    border-radius: 24px;
    padding: 3.5rem;
    backdrop-filter: blur(20px);
    box-shadow: 0 20px 50px rgba(0, 0, 0, 0.3);
    margin-bottom: 2rem;
  }
  .intro-title {
    font-family: var(--font-heading);
    font-size: 3rem;
    font-weight: 800;
    margin-bottom: 1rem;
    letter-spacing: -0.5px;
  }
  .intro-desc {
    font-size: 1.2rem;
    color: var(--text-secondary);
    margin-bottom: 3rem;
    line-height: 1.6;
  }
  .pillars-grid {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 1.5rem;
```

```

        margin-bottom: 3.5rem;
    }
    .pillar-card {
        background: rgba(255, 255, 255, 0.02);
        border: 1px solid rgba(255, 255, 255, 0.05);
        border-radius: 16px;
        padding: 1.5rem;
        transition: var(--transition-smooth);
    }
    .pillar-card:hover {
        transform: translateY(-5px);
        border-color: var(--card-hover-border);
        background: rgba(0, 242, 254, 0.02);
    }
    .pillar-icon {
        font-size: 2rem;
        margin-bottom: 1rem;
    }
    .pillar-title {
        font-family: var(--font-heading);
        font-size: 1.1rem;
        font-weight: 700;
        margin-bottom: 0.5rem;
    }
    .pillar-desc {
        font-size: 0.9rem;
        color: var(--text-secondary);
        line-height: 1.4;
    }
    .cta-btn {
        font-size: 1.1rem;
        padding: 1rem 2.5rem;
        border-radius: 12px;
        text-decoration: none;
        display: inline-flex;
        align-items: center;
        gap: 0.75rem;
        font-weight: 600;
        box-shadow: var(--glow-primary);
    }
    .footer-note {
        color: var(--text-muted);
        font-size: 0.85rem;
        margin-top: 1rem;
    }
}
</style>
</head>
<body>
    <!-- Background Glow Spheres -->
    <div class="glow-sphere sphere-1"></div>
    <div class="glow-sphere sphere-2"></div>

```

```

<div class="glow-sphere sphere-3"></div>

<div class="home-container">
  <div class="intro-card">
    <div class="brand-logo" style="justify-content: center; margin-bottom: 1.5rem;">
      <i class="fa-solid fa-chart-line logo-icon" style="font-size: 2rem;"></i>
      <span class="logo-text" style="font-size: 2rem;">Vanguard <span class="accent-
text">HDI</span></span>
    </div>

    <h1 class="intro-title">Human Development Index Forecasting</h1>
    <p class="intro-desc">
      Welcome to Vanguard HDI, a premium predictive analytics platform. Leverage
      machine learning algorithms trained on decades of global developmental data to predict and
      evaluate national human development scores.
    </p>

    <div class="pillars-grid">
      <div class="pillar-card">
        <i class="fa-solid fa-heart-pulse pillar-icon" style="color: #00F2FE;"></i>
        <h3 class="pillar-title">Long & Healthy Life</h3>
        <p class="pillar-desc">Measured by Life Expectancy at Birth, capturing
country-level healthcare standards and quality of life.</p>
      </div>
      <div class="pillar-card">
        <i class="fa-solid fa-graduation-cap pillar-icon" style="color: #7F00FF;"></i>
        <h3 class="pillar-title">Knowledge & Education</h3>
        <p class="pillar-desc">Assessed via Expected and Mean Years of Schooling,
reflecting educational opportunity and literacy rates.</p>
      </div>
      <div class="pillar-card">
        <i class="fa-solid fa-dollar-sign pillar-icon" style="color: #38EF7D;"></i>
        <h3 class="pillar-title">Decent Living Standard</h3>
        <p class="pillar-desc">Estimated through Gross National Income (GNI) per
Capita (PPP $), mapping purchasing power parity and economic health.</p>
      </div>
    </div>

    <a href="/dashboard" class="btn btn-primary cta-btn">
      <span>Launch Interactive Workspace</span>
      <i class="fa-solid fa-compass"></i>
    </a>

    <p class="footer-note">Powered by a pre-trained scikit-learn Linear Regression
model.</p>
  </div>
</div>
</body>
</html>

```

Predict.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>HDI Prediction Result - Vanguard HDI</title>
  <!-- Google Fonts -->
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link
    href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700&family=Outfit:wght@400;600;700;800&display=swap" rel="stylesheet">
  <!-- FontAwesome for Premium Icons -->
  <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css">
  <!-- Custom Style Sheet -->
  <link rel="stylesheet" href="/static/index.css">
<style>
  .result-container {
    display: flex;
    flex-direction: column;
    align-items: center;
    justify-content: center;
    min-height: 100vh;
    padding: 2rem;
    max-width: 600px;
    margin: 0 auto;
    text-align: center;
    position: relative;
    z-index: 10;
  }
  .result-card {
    background: var(--card-bg);
    border: 1px solid var(--card-border);
    border-radius: 24px;
    padding: 3rem;
    backdrop-filter: blur(20px);
    box-shadow: 0 20px 50px rgba(0, 0, 0, 0.3);
    width: 100%;
  }
  .score-circle {
    width: 180px;
    height: 180px;
    border-radius: 50%;
```

```

    background: rgba(0, 242, 254, 0.05);
    border: 3px solid #00F2FE;
    display: flex;
    flex-direction: column;
    align-items: center;
    justify-content: center;
    margin: 2rem auto;
    box-shadow: var(--glow-primary);
}
.score-val {
    font-family: var(--font-heading);
    font-size: 3rem;
    font-weight: 800;
    color: #00F2FE;
}
.score-label {
    font-size: 0.75rem;
    font-weight: 700;
    color: var(--text-primary);
    letter-spacing: 1px;
    margin-top: 5px;
}
}
.metrics-summary {
    background: rgba(255, 255, 255, 0.02);
    border: 1px solid rgba(255, 255, 255, 0.05);
    border-radius: 16px;
    padding: 1.5rem;
    margin-bottom: 2.5rem;
    text-align: left;
}
}
.metric-row {
    display: flex;
    justify-content: space-between;
    padding: 0.5rem 0;
    border-bottom: 1px solid rgba(255, 255, 255, 0.05);
}
}
.metric-row:last-child {
    border-bottom: none;
}
}
.metric-label {
    color: var(--text-secondary);
    font-size: 0.95rem;
}
}
.metric-val-text {
    font-weight: 600;

```

```

        color: var(--text-primary);
    }
</style>
</head>
<body>
    <!-- Background Glow Spheres -->
    <div class="glow-sphere sphere-1"></div>
    <div class="glow-sphere sphere-2"></div>
    <div class="glow-sphere sphere-3"></div>

    <div class="result-container">
        <div class="result-card">
            <div class="brand-logo" style="justify-content: center; margin-bottom: 1rem;">
                <i class="fa-solid fa-chart-line logo-icon"></i>
                <span class="logo-text">Vanguard <span class="accent-text">HDI</span></span>
            </div>

            <h2 style="font-family: var(--font-heading); font-weight: 700;">Prediction Result</h2>
            <p style="color: var(--text-secondary); font-size: 0.95rem;">Calculated HDI score using pre-trained Linear Regression.</p>

            <div class="score-circle">
                <span class="score-val">{{ "%.4f" | format(score) }}</span>
                <span class="score-label">HDI SCORE</span>
            </div>

            <div class="metrics-summary">
                <div class="metric-row">
                    <span class="metric-label"><i class="fa-solid fa-heart-pulse" style="color: #00C6FF; margin-right: 8px;"></i> Life Expectancy</span>
                    <span class="metric-val-text">{{ life_expectancy }} yrs</span>
                </div>
                <div class="metric-row">
                    <span class="metric-label"><i class="fa-solid fa-graduation-cap" style="color: #F9D423; margin-right: 8px;"></i> Mean Schooling</span>
                    <span class="metric-val-text">{{ mean_schooling }} yrs</span>
                </div>
                <div class="metric-row">
                    <span class="metric-label"><i class="fa-solid fa-graduation-cap" style="color: #FF4E50; margin-right: 8px;"></i> Expected Schooling</span>
                    <span class="metric-val-text">{{ expected_schooling }} yrs</span>
                </div>
                <div class="metric-row">
                    <span class="metric-label"><i class="fa-solid fa-dollar-sign" style="color:

```

```

#B3FFAB; margin-right: 8px;"></i> GNI per Capita</span>
    <span class="metric-val-text">${ { " {:.0f} ".format(gni) } }</span>
</div>
</div>

    <a href="/dashboard" class="btn btn-primary" style="display: inline-flex; align-
items: center; gap: 8px; font-weight: 600; width: 100%; justify-content: center;
padding: 0.85rem 1.5rem; border-radius: 10px;">
    <i class="fa-solid fa-arrow-left"></i>
    <span>Back to Workspace</span>
</a>
</div>
</div>
</body>
</html>

```

11.2 Dataset Link

<https://github.com/Guided-Projects/HumanDevelopmentIndex/tree/main/Dataset>

11.3 GitHub Repository Link

<https://github.com/bellalaprameelapallavi/HDI.git>

11.4 Project Demo Link

<https://drive.google.com/drive/folders/17-NNetDTSZUz0l27tOd9UvJPSbWzpS2G>